THE IRON NEXUS

Project Report · Data Structures · Spring 2026

GROUP MEMBERS

Shahram Shafiq	24i-2541	Section B
Haris Said	24i-0527	Section B
Haris Shafiq	24i-0746	Section B

OVERVIEW

Our project named the iron nexus is a CLI based railway management system which was coded in C++, no STL containers were used as we aren't allowed to use them. All the data structures are implemented from scratch they are the trees, lists, hash tables and graphs, we used pointers. Our system handles train registration, coach management, a station network, booking for seats and operation logging and saving to files and reading from them so that we can reuse data even if the program ends and we restart it later.

CORE MODULES

Module 1 - Train Registry · AVL Tree

We used AVL tree and its key was the train ID, we could've used a BST but that only works if the insertions are random. Train IDs however are assigned in a sequential manner but a BST with sorted insertion starts imitating linked lists and lookup complexity goes up to O(n). The AVL however will fix balance factor after an insertion or deletion and keeps the height O(log n), regardless of the sequence in which we do IDs insertions. At bigger levels like 10,000 trains, this difference is a huge deal.

Module 2 - Coach Management · Doubly Linked List

Each coach in the train is a part of a doubly linked list. Our requirement was O(1) insertion complexity at both the ends as the VIP coaches and economy ones move in reverse, traversing in both directions for staff who are walking along the train and also for a clean reverse on the return journeys. A singly linked list would make the backward movement very complicated. We have also added a small optimization when inserting, which is that we check which end is closer and we start the walk from there so basically on average, we only traverse half of the list.

Module 3 - Railway Network · Adjacency List + Dijkstra

The station network is an implementation of a weighted undirected graph which is stored as an adjacency list. We didn't use a matrix because in real, the networks of railways are sparse so it would result in mostly 0s in a 15x15 matrix. Dijkstra will find the shortest path whether in the context of distance or travel-time mode, it depends on what the operator needs. Both share the same function but we have a flag to switch between the weights. To check for connectivity, we use BFS with an array-based implementation of queues.

Module 4 - Seating Chart · Hash Table + BST (Dual Layer)

The seating system however uses two data structures in its implementation. First one is hash table with linear probing, multiplicative hashing and a table which has a prime number size, it handles booking, lookup and cancellation in a complexity of O(1). The BST is used for display, so when a staff needs a sorted list of booked or available seats, in order traversal will give that without any need to do extra sorting. Both the data structures are always in sync for every booking or cancellation.

Module 5 - Operation Log · Stack

Every action that is performed, we push it into a linked list-based implementation of a stack. Operators usually care about the recent activities, so LIFO of a stack fits in here as we won't need to do any sorting. Each entry stores the action in text format, the module which triggered it and also a timestamp.

BONUS FEATURES

Undo / Redo: It was built using a 2-stack pattern. Mutating actions will get pushed into the undo stack. Undoing it will reverse the action and it will be pushed into a redo stack. If we start a new action, the redo stack is completely cleared.

File I/O. The AVL tree data structure saved in a pre-order way so rebuilding it is simply inserting it again as the balance restores by itself on the way back. The network file saves one direction per edge (source < destination) to avoid writing each edge twice. As an extra, the seating BST also gets saved in a hierarchical file that makes sure the tree structure remains intact.

Colored Output. Uses ANSI escape codes with Windows virtual terminal processing enabled at startup.